Architectural Paradigms and Latency Optimization Strategies for High-Throughput Extractive Non-LLM Retrieval-Augmented Generation

Executing Retrieval-Augmented Generation (RAG) pipelines within strict sub-200 millisecond Service Level Agreements (SLAs) across datasets containing hundreds of thousands to millions of entries requires eliminating the primary latency bottleneck of modern conversational AI: autoregressive Large Language Model (LLM) generation. Large-scale generative decoders process tokens sequentially, incurring linear time complexity relative to output sequence length, which frequently pushes total end-to-end execution latency well beyond 1,000 milliseconds.

To achieve deterministic sub-200ms processing on datasets exceeding 200,000 rows, production architecture must pivot toward non-LLM extractive RAG pipelines[1]. These pipelines substitute generative decoders with high-precision bi-encoder embedding models, graph-accelerated vector indices, quantized late-interaction re-rankers, and static extractive neural span readers (e.g., fine-tuned BERT/RoBERTa architectures optimized via custom execution engines)[3].

Pipeline Latency Budget Allocation

To maintain strict latency conformance across a corpus exceeding 200,000 records, the end-to-end execution window of 200 milliseconds must be partitioned across the discrete operational stages of the extractive pipeline. The overall pipeline latency T_{total} is formally expressed as:

$$T_{\text{total}} = T_{\text{cache}} + (1 - H_{\text{rate}}) \times (T_{\text{embed}} + T_{\text{retrieval}} + T_{\text{rerank}} + T_{\text{extract}}) + T_{\text{overhead}}$$

Where H_{rate} represents the semantic cache hit rate, T_{cache} is the cache lookup time, T_{embed} is the neural bi-encoder query vectorization time, $T_{\text{retrieval}}$ is the vector store Approximate Nearest Neighbor (ANN) search time, T_{rerank} is the cross-encoder or late-interaction scoring latency, T_{extract} is the extractive neural reader span-detection time, and T_{overhead} encompasses network serialization and framework inter-process communication (IPC).

Pipeline Phase	Core Mechanism	Target P50 Latency (ms)	Target P99 Latency (ms)	Hardware Execution Context
1. Cache Lookup	In-Memory / Vector Semantic Cache	1.5	5.0	Redis / Shared Memory
2. Query Embedding	Quantized Bi-Encoder (INT8 / FP16)	12.0	25.0	GPU (TensorRT) / CPU (ONNX)
3. ANN Vector Retrieval	HNSW Graph Search with Quantization	4.0	12.0	Distributed Vector DB / RAM
4. Passage Reranking	Cross-Encoder / ColBERT Late-Interaction	25.0	45.0	TensorRT / OpenVINO Execution
5. Extractive Span QA	Extractive Neural Reading (DistilBERT/RoBERTa)	35.0	75.0	Dedicated Inference Engine
6. IPC & Serialization	Zero-Copy gRPC Protocol Buffers	1.5	5.0	Host Memory / Local Network
Total Cold-Path Execution	**Complete Non-LLM Pipeline**	**79.0 ms**	**167.0 ms**	**End-to-End Budget Conformance**

Architectural Implementation Methods for Extractive Non-LLM Systems

1. Two-Stage Coarse-to-Fine Quantized Vector Search with Native Rescoring

To query large dataset spaces without triggering prohibitive memory-bandwidth stalls, candidate retrieval relies on a two-tier vector evaluation scheme[3]. Dense embeddings generated by the query encoder are initially compressed using 1-bit Binary Quantization (BQ) or 8-bit Scalar Quantization (SQ8)[3]. Candidate generation executes an approximate nearest neighbor search over the quantized index using SIMD-accelerated Hamming distance or integer dot products, retrieving an over-sampled candidate pool ($K = 100$)[7].
Subsequently, a full-precision floating-point (FP32 or FP16) rescoring pass is executed exclusively over the retrieved candidate IDs using uncompressed memory-mapped vectors[3]. This structural decoupling reduces initial graph navigation latency to under 5ms while recovering over 96% of full-precision retrieval accuracy[3]. Originating from early high-dimensional multimedia indexing, this approach has evolved into the standard operating mode for vector databases handling high-concurrency workloads[8].

2. Multi-Tier Hybrid Lexical-Dense Vector Indexing with Late-Interaction Filtering

Pure dense retrieval struggles with exact keyword hits such as serial numbers or specific entities, while pure lexical retrieval (BM25) fails to capture abstract semantics[1]. This implementation constructs a parallelized hybrid candidate generation engine[3]. Incoming queries are dispatched concurrently to a high-speed BM25 inverted index and an HNSW dense vector store[3].
The top candidates from both channels are fused via Reciprocal Rank Fusion (RRF) before being passed to a late-interaction model such as ColBERT utilizing the PLAID engine[4]. Late interaction computes token-level vector similarities via max-sim operations rather than full cross-attention matrix multiplications, maintaining sub-30ms re-ranking latency while matching cross-encoder-level retrieval precision[3].

3. Matryoshka Representation Learning (MRL) Dynamic Truncation Retrieval

Matryoshka Representation Learning trains neural embedding models to nest lower-dimensional vector representations within higher-dimensional outputs[3]. Instead of conducting similarity searches across 1,536 or 4,096 dimensions, the system extracts and indexes only the leading 128 or 256 dimensions[3].
Initial candidate retrieval operates on these truncated vectors, accelerating vector dot-product execution and minimizing CPU cache misses[3]. Once the top 30 candidate documents are identified, the system reads the full 1,536-dimensional slice for those specific vectors to refine the top-5 ranking, securing significant latency reductions during initial index traversal without requiring separate embedding models for coarse and fine phases[3].

4. Hierarchical Semantic Caching with Intent-Based Vector Routing

By placing an intelligent semantic caching layer prior to vector search and neural reading, identical or semantically equivalent queries bypass the retrieval and extraction pipeline entirely[14]. Incoming query strings are embedded using a lightweight bi-encoder model and checked against an in-memory vector cache holding recent query-response pairs[14].
If the cosine similarity between the incoming query vector and a cached query vector exceeds a calibrated threshold ($\cos\theta \geq 0.92$), the pre-extracted answer payload is returned immediately[15]. This architecture yields sub-10ms response times for high-frequency queries, smoothing system throughput spikes and reducing overall hardware utilization[14].

5. Contrastive Sparse Representation (CSRv2) Ultra-Sparse Retrieval

Contrastive Sparse Representation (CSRv2) maps continuous dense representations into high-dimensional, highly sparse vector spaces where only a minute fraction ($k = 2$ or $k = 4$) of dimensions contain non-zero values[13].
By deploying ultra-sparse embeddings, candidate generation operates through sparse inverted index structures rather than graph structures[13]. This yields up to a $7\times$ speedup over Matryoshka-truncated dense retrieval and up to $300\times$ improvement in computational efficiency compared to full dense models, achieving sub-millisecond candidate generation across datasets exceeding 200,000 rows[13].

6. ONNX-Engineered Extractive Neural Span Selection

Replacing autoregressive LLMs with neural Extractive Question Answering (QA) models (e.g., fine-tuned SQuAD-style RoBERTa, DeBERTa, or DistilBERT variants) shifts the answer generation phase from dynamic token generation to static span classification[5]. The extractive reader takes the query and top-retrieved document contexts, predicting start and end token indices via parallel linear classification heads:

$$P_{\text{start}}(i) = \frac{e^{\mathbf{w}_s \cdot \mathbf{h}_i}}{\sum_k e^{\mathbf{w}_s \cdot \mathbf{h}_k}}, \quad P_{\text{end}}(j) = \frac{e^{\mathbf{w}_e \cdot \mathbf{h}_j}}{\sum_k e^{\mathbf{w}_e \cdot \mathbf{h}_k}}$$

Where $\mathbf{h}_i$ represents the contextualized hidden state of token i, and $\mathbf{w}_s, \mathbf{w}_e$ represent learned start and end classification vectors. When compiled into ONNX Runtime or TensorRT formats with fixed dynamic axes and INT8 precision, the extraction phase completes within 20ms to 40ms, bypassing generation latencies entirely[5].

7. In-Memory Graph Indexing with Memory-Mapped Partitioning

To manage memory costs while preserving sub-10ms search performance, vector database engines utilize a decoupled storage strategy[8]. The structural topology of the Hierarchical

Navigable Small World (HNSW) graph is pinned permanently inside RAM to ensure navigation paths execute without IO blocking[8].
Concurrently, the raw high-dimensional vector payloads and associated text metadata are mapped to storage via memory-mapped files (memmap)[8]. The operating system page cache dynamically handles chunk loading only when specific vector IDs require precision rescoring, optimizing RAM utilization while maintaining high traversal throughput[8].

8. Semantic ID (SID) and Finite Scalar Quantization Codebook Retrieval

Instead of performing distance calculations across continuous vector spaces, Finite Scalar Quantization (FSQ) and Semantic IDs (SIDs) quantize continuous embeddings into discrete structural codebook indices[22].
The vector space is bounded into fixed categorical buckets per dimension, generating a compact integer tuple (Semantic ID) for each document[22]. Retrieval shifts from floating-point similarity calculations to exact integer lookups and hash-table intersections, enabling high-throughput candidate extraction in sub-5ms frames[22].

9. Asynchronous Concurrent Search and Embedding Pipeline Execution

Pipeline execution latency increases when operational stages run sequentially. The asynchronous concurrent architecture executes non-dependent operations in parallel using async event loops and non-blocking worker threads[2].
While the bi-encoder computes the query vector, metadata filters and primary database keys are prefetched concurrently based on request headers or user context[2]. Similarly, candidate document fetching and tokenizer preprocessing for the neural reader overlap with re-ranking score updates, trimming significant host-side execution overhead[2].

Pipeline Execution Stage	Sequential Execution Latency (ms)	Asynchronous Concurrent Latency (ms)	Concurrent Overlap Strategy
Query Processing & Embedding	35.0	35.0	Executed on main worker stream
Metadata Prefetching	20.0	0.0 (Overlapped)	Concurrent async task during embedding
ANN Candidate Retrieval	15.0	15.0	Graph navigation via pre-fetched filters

Tokenizer Preprocessing	15.0	0.0 (Overlapped)	Parallel execution during candidate fetches
Neural Span Extraction	45.0	45.0	Execution engine host-pinned streams
Total Pipeline Overhead	**130.0 ms**	**95.0 ms**	**26.9% Pipeline Execution Speedup**

10. Integrated Payload Masking for Pre-Filtered Graph Navigation

Standard post-filtering strategies perform ANN search over the entire vector space and subsequently discard candidates that fail payload criteria (e.g., tenant ID, date ranges)[23]. This causes latency spikes and candidate starvation if the top matches are filtered out[23].
Pre-filtered graph navigation integrates metadata constraints directly into the HNSW traversal loop[8]. As the search algorithm navigates graph edges, it checks a bitset mask representing valid metadata IDs[23]. Invalid nodes are bypassed during traversal, ensuring that vector distance calculations are performed exclusively on compliant nodes, stabilizing latency below 10ms regardless of filter selectivity[8].

Latency Efficiency Techniques for Sub-200ms Execution

1. Scalar Quantization (INT8) and Quantization-Aware Training (QAT)

Scalar Quantization (SQ8) transforms 32-bit floating-point vector dimensions ($float32$) into 8-bit integers ($int8$) by mapping values across a calibrated min/max range[3]. This drops vector memory footprint by 75% and replaces floating-point Multiply-Accumulate (MAC) operations with fast SIMD integer instructions[3].

$$\text{Quantize}(x) = \text{Clamp}\left(\text{Round}\left(\frac{x - \min}{\max - \min} \times 255\right), 0, 255\right)$$

Applying Quantization-Aware Training (QAT) or fine-tuned scale calibration ensures that accuracy loss remains under 1%, while vector database search speeds accelerate by up to $40\times$ [11].

2. Static Graph Optimization and Operator Fusion via ONNX Runtime

Deploying neural models directly in PyTorch incurs dynamic execution graph overhead and interpreter context switching, leading to inference latencies up to 2.0 seconds[5]. Exporting models to static Open Neural Network Exchange (ONNX) format enables offline execution graph optimizations[5].

ONNX Runtime fuses adjacent matrix operations (such as GELU activation fusion, LayerNormalization folding, and Multi-Head Attention key-value projection concatenation), eliminating redundant memory transfers and reducing neural embedding latency from over 1,000ms down to sub-30ms[5].

3. Thread Pool Affinity Tuning and Intra-Op Thread Spinning Control

For CPU-based retrieval and extraction pipelines, unmanaged thread contention causes severe latency jitter. Tuning ONNX Runtime performance requires configuring intra-op (intra_op_num_threads) and inter-op thread pools[25].

Binding execution threads to specific physical CPU cores using hardware affinity masks (-T configurations) prevents cross-socket NUMA bus hops[27]. Furthermore, disabling thread spinning between inference executions (-Z and -D flags) prevents idle worker threads from saturating memory bandwidth, maintaining consistent sub-20ms model runs[25].

4. Hardware-Native Engine Compilation (TensorRT and OpenVINO)

To extract maximum performance from underlying hardware, neural models (bi-encoders and extractive readers) are compiled into target-specific binary engines[28]. NVIDIA TensorRT compiles ONNX models into optimized CUDA execution plans, leveraging FP16/INT8 Tensor Cores and auto-tuning kernel selection for the specific GPU architecture[21].

On Intel/ARM CPU architectures, compiling via OpenVINO or leveraging AVX-512 / AMX instruction sets achieves parallel matrix calculations[25]. Hardware compilation drops neural inference latency by $2\times$ to $5\times$ compared to unoptimized runtimes[5].

5. Zero-Copy Memory Management via Host and Device I/O Binding

In standard ML serving pipelines, input arrays are copied from Host RAM to Python objects, serialized to C++ pointers, transferred over PCIe to GPU VRAM, processed, and copied back to Host RAM[21]. This data movement can consume up to 30% of total request latency[21].

Implementing ONNX Runtime or TensorRT I/O Binding pre-allocates contiguous device memory buffers for input token IDs and output logit tensors[21]. Inference engines read from and write directly to pinned shared memory handles, eliminating memory copy overhead[21].

6. SIMD-Accelerated Bitwise Popcount Distance Metrics

When vectors are quantized to 1-bit binary representations, the distance calculation between query vector $\mathbf{q}$ and document vector $\mathbf{d}$ transforms from floating-point Euclidean distance to a bitwise XOR followed by a population count (popcount) operation[7]:

$$D_{\text{Hamming}}(\mathbf{q}, \mathbf{d}) = \text{popcount}(\mathbf{q} \oplus \mathbf{d})$$

Modern x86 (AVX-512) and ARM (NEON) processors execute vector popcount commands natively across thousands of bits per CPU cycle[25]. This accelerates candidate retrieval across 200,000 rows to under 2ms, enabling fast search on commodity hardware[7].

7. Context Bounding and Dynamic Passage Token Truncation

Extractive neural reader latency scales quadratically $O(N^2)$ with input sequence token length N due to self-attention matrix evaluations. Naive RAG implementations pass large multi-thousand-token document contexts to the reader, degrading execution speeds[1]. Context bounding dynamically truncates candidate passages into precise 256-token or 300-token windows centered around the dense vector match density[2]. Restricting the extractive reader's input window to $N \leq 300$ bounds matrix computation overhead, keeping extraction processing strictly under 40ms[2].

8. Native Kernel Page Cache Management with Memory-Mapped Storage (memmap)

When dataset scales exceed available physical RAM, relying on standard file I/O operations introduces high-latency disk reads. Configuring vector stores to utilize memory-mapped files (on_disk: true or memmap) maps the vector dataset directly into the virtual address space of the process[8].

The operating system kernel automatically manages paging, utilizing available free RAM as an elastic page cache[8]. Frequently accessed vector indices remain hot in memory, while cold vectors reside on storage, achieving near in-memory search latencies at reduced hardware costs[8].

9. Multi-Tiered In-Memory Caching Topology

To maximize cache throughput, system memory should be structured into a three-tiered cache topology based on query access patterns[18]:

- **Tier 1 (Hot Local Cache):** A zero-latency process-memory LRU cache holding the top 1,000 frequent queries. Lookups resolve in 1ms to 2ms[18].
- **Tier 2 (Warm Shared Cache):** A centralized Redis instance storing up to 50,000 vector embeddings and pre-computed extractive responses. Lookups resolve in 5ms to 10ms[14].
- **Tier 3 (Cold Vector Search):** The vector database and neural reader pipeline, activated only upon cache misses[18].

10. Asynchronous CUDA Event Streams and Non-Blocking GPU Pipeline Execution

On GPU-accelerated serving nodes, sequential kernel launches cause GPU starvation while

waiting for CPU control instructions[21]. Utilizing asynchronous CUDA event streams allows the CPU to enqueue query embedding kernels, late-interaction re-ranking operations, and neural extraction passes into concurrent GPU queues[2].
The GPU executes these operations seamlessly without waiting for host synchronization signals, maximizing Tensor Core utilization and cutting pipeline overhead down to hardware execution limits[21].

Performance Benchmarks and Architectural Synthesis

To evaluate the operational suitability of vector engine infrastructure for sub-200ms non-LLM pipelines, the following performance data compares vector engines across datasets exceeding 1,000,000 vectors under standardized latency constraints[23].

Vector Database Engine	Indexing Topology	Quantization Support	P50 Search Latency (ms)	P99 Search Latency (ms)	Throughput (RPS)	Payload Filtering Strategy
Qdrant	Custom HNSW	SQ, PQ, BQ	3.54	8.62	1,238.0	Single-Stage Pre-filtering
Weaviate	Dynamic HNSW	SQ, PQ	4.99	11.33	1,142.1	Bitset Mask Pre-filtering
Elasticsearch	HNSW / Lucene	SQ	22.10	135.68	716.8	Iterative Post-filtering
Redis Search	HNSW / Flat	SQ	140.65	167.35	625.2	Attribute Pre-filtering
Milvus	HNSW / DiskANN	PQ, SQ	393.31	576.65	219.1	Two-Pass Filtering

The benchmark data indicates that vector engines utilizing native Rust or C++ HNSW implementations with single-stage pre-filtering (such as Qdrant or Weaviate) maintain P99

latencies well below 15ms[20]. Engines reliant on post-filtering or unoptimized runtime layers experience long-tail latency degradation under concurrent filter loads, exceeding the target SLAs[23].

Architectural optimization for low-latency non-LLM retrieval involves structural trade-offs across memory compaction, recall precision, and hardware execution efficiency. Compressing vector spaces via 1-bit Binary Quantization or INT8 Scalar Quantization yields dramatic reductions in memory bandwidth consumption, enabling multi-thousand-query throughput on sub-10ms search budgets[7]. However, aggressive quantization introduces precision degradation during initial candidate selection[3].

Systems resolve this precision loss by over-sampling candidate pools ($K = 50$ or $K = 100$) during coarse graph navigation and subsequently applying full-precision floating-point rescoring or late-interaction MaxSim scoring exclusively over the retrieved candidate IDs[3].

Furthermore, replacing autoregressive language models with ONNX-compiled extractive span readers transforms text generation into a single-pass matrix classification task, eliminating dynamic decoding loops and bounding extraction latency within a 20ms to 40ms window[5]. Combined with zero-copy I/O binding, CPU core affinity pinning, and non-blocking asynchronous CUDA event streams, this design ensures that total pipeline execution stays well under the 200 millisecond SLA while processing corpora exceeding 200,000 records[2].

Production Deployment Blueprint for Sub-200ms SLAs

To deploy an extractive non-LLM RAG system handling >200,000 records within a strict 200ms processing SLA, systems should be deployed according to the following operational pipeline:

1. **Document Ingestion:** Compute 1,536-dimensional dense embeddings for document chunks capped at $N \leq 300$ tokens[1]. Quantize vectors to INT8 precision using Scalar Quantization, maintaining the HNSW graph topology in RAM while leveraging memmap for payload storage[8].
2. **Caching Gateway:** Implement an in-memory Redis vector cache using a cosine similarity threshold ($\cos\theta \geq 0.92$) to serve semantically equivalent queries within 10ms, bypassing vector search and neural extraction[14].
3. **Query Vectorization:** Compile the bi-encoder embedding model to ONNX format, optimized via TensorRT or OpenVINO using INT8 quantization and I/O Binding to execute query vectorization in under 25ms[5].
4. **Graph Candidate Retrieval:** Execute single-stage pre-filtered HNSW graph navigation over INT8 vectors, extracting the top $K = 50$ candidate IDs within 10ms[8].
5. **Passage Re-Ranking:** Pass candidate IDs to a late-interaction ColBERT/PLAID engine or

an INT8 cross-encoder to isolate the top $M = 5$ context passages within 35ms[3].

6. **Neural Span Extraction:** Input the query and top 5 context passages into an ONNX-compiled Extractive Reader (e.g., RoBERTa/DistilBERT QA) to locate and extract answer spans within 40ms[2].

This technical configuration eliminates autoregressive generation overhead, securing deterministic sub-200ms execution while maintaining high extraction accuracy across large-scale document collections[1].

Works cited

1. RAG Without the Lag: Interactive Debugging for Retrieval-Augmented Generation Pipelines, https://arxiv.org/html/2504.13587v1
2. Reduce RAG Latency: From 2000ms to 200ms, https://app.ailog.fr/en/blog/guides/reduce-rag-latency
3. HAKARI-Bench: A Lightweight Benchmark for Comparing Retrieval Architectures and Efficiency Settings under Unified Conditions - arXiv, https://arxiv.org/html/2606.22778v1
4. Late Interaction & ColBERT · Retrieval Systems · AI Daddy, https://www.aidaddy.tech/learn/06-retrieval-systems/11-late-interaction-colbert
5. ONNX Runtime for Production ML: Optimize Model Inference Speed - Reintech, https://reintech.io/blog/onnx-runtime-production-ml-optimizing-model-inference-speed
6. From 2s to 600ms: PyTorch vs ONNX Runtime | by Naoki Goto - Medium, https://medium.com/@naokig/from-2s-to-600ms-pytorch-vs-onnx-runtime-5fb2ef14e4f0
7. Optimization of embeddings storage for RAG systems using quantization and dimensionality reduction techniques. - arXiv, https://arxiv.org/html/2505.00105v1
8. Vector Search Resource Optimization Guide - Qdrant, https://qdrant.tech/articles/vector-search-resource-optimization/
9. BitNet Text Embeddings - arXiv, https://arxiv.org/html/2606.25674v2
10. HNSW_PQ | Milvus Documentation, https://milvus.io/docs/hnsw-pq.md
11. Qdrant High-Performance Vector Search Engine, https://qdrant.tech/qdrant-vector-database/
12. HAKARI-Bench: A Lightweight Benchmark for Comparing Retrieval Architectures and Efficiency Settings under Unified Conditions - arXiv, https://arxiv.org/pdf/2606.22778
13. CSRv2: Unlocking Ultra-Sparse Embeddings - arXiv, https://arxiv.org/pdf/2602.05735
14. How to optimize machine learning inference costs and performance - Redis, https://redis.io/blog/machine-learning-inference-cost/
15. Semantic Caching: Boost LLM Speed & Reduce Costs - Truefoundry, https://www.truefoundry.com/blog/semantic-caching
16. LLM Token Optimization: Cut Costs & Latency in 2026 - Redis, https://redis.io/blog/llm-token-optimization-speed-up-apps/

17. https://redis.io/blog/machine-learning-inference-cost/#:~:text=Redis%3A%20One%20platform%20for%20semantic%20caching%20and%20vector%20search&text=Vector%20embeddings%20live%20alongside%20your,stores%20new%20embeddings%20for%20misses.
18. Semantic Cache: The Smartest Way to Speed Up RAG (Without More GPUs), https://www.codebrains.co.in/blog/2025/ai/semantic-cache-smartest-way-to-speed-up-rag
19. Real-time decisioning - Redis, https://redis.io/solutions/real-time-decisioning/
20. Qdrant vs Milvus: Which Vector Database Should You Choose? - F22 Labs, https://www.f22labs.com/blogs/qdrant-vs-milvus-which-vector-database-should-you-choose/
21. Performance Benchmarking — NVIDIA TensorRT, https://docs.nvidia.com/deeplearning/tensorrt/latest/performance/benchmarking.html
22. SIDE: Semantic ID Embedding for effective learning from sequences - arXiv, https://arxiv.org/html/2506.16698v1
23. Vector Search Benchmarks - Qdrant, https://qdrant.tech/benchmarks/
24. Bekko Embedding: Parameter-Efficient Multilingual Retrieval with Ultra-Compact Encoders, https://arxiv.org/html/2607.25180
25. Benchmark ONNX runtime performance with onnxruntime_perf_test - Arm Learning Paths, https://learn.arm.com/learning-paths/servers-and-cloud-computing/onnx-on-azure/benchmarking/
26. Tune performance - ONNX Runtime, https://onnxruntime.ai/docs/performance/tune-performance/
27. How to measure the performance of your models using ONNX Runtime - stm32mpu - ST wiki, https://wiki.st.com/stm32mpu/wiki/How_to_measure_the_performance_of_your_models_using_ONNX_Runtime
28. Benchmarking Edge Inference Strategies for Deep Learning Models in Industrial Machine Vision The research leading to these results has received funding from “Proyecto Desarrollo de una Estrategia integral de REciCladO de BATerías - RECOBATS”, as part of the TransMisiones 2024 initiative, under project file number PLEC2024-011135. This project is funded by the Spanish Ministry of Science - arXiv, https://arxiv.org/html/2607.11356v1